

Station ALMA-7, Part II: The Teleporter Incident

Enums · Structs · Classes · Properties · Access Control

Two weeks after the rescue. ALMA-7 is running again, and the crew installed a short-range teleporter to move people and cargo between decks. It worked for nine days.

On day ten the teleporter reported a successful transfer of the entire crew. The manifest said every crate arrived. The oxygen logs said nothing changed. All three reports were wrong, and one crew member now appears on two decks at once.

The station computer doesn't have a hardware problem. It has a **type** problem. Somebody used a class where a struct was needed, and a struct where a class was needed. Your job is to rebuild the manifest system with the right types, prove the difference in code, and produce the **integrity code** that certifies the station is consistent again.

Topics: enums (raw values, associated values, `CaseIterable`), structs vs classes, value vs reference semantics, initializers, stored / lazy / computed properties, property observers, instance and type methods, `mutating`, access control.

Submission: your completed `ALMA7_Part2_Starter.swift` (or a Playground with the same contents), with answers to the defense questions in the comment block at the end.

Station Rules

1. **! (force unwrap) is still forbidden.** -0.5 points each. The `Incident Report` section is the only exception.
2. Still no `map`, `filter`, `reduce`, or `compactMap`. Write the loops yourself.
3. Use the **exact** type, case, and function names given here.
4. Every type you define must be exercised by at least two `print` calls.
5. **Default to struct.** Use `class` only where this assignment requires it — and in the defense questions, be ready to say why.

Starter File

Download `ALMA7_Part2_Starter.swift` and paste its contents into a new Playground (or run `swift ALMA7_Part2_Starter.swift`). **Do not modify the starter section.** Do not copy code from this PDF.

The starter file gives you:

Item	What it is
<code>fields(_:separatedBy:)</code>	Splits a line into <code>[String]</code> — your parsing helper from Module 3
<code>rawManifest</code>	10 raw cargo lines, some of them corrupted
<code>deckReadings</code>	6 oxygen readings as <code>(deck: String, oxygen: Int)</code> — one deck name is not real
<code>crewData</code>	4 crew records as <code>(name: String, deck: String, oxygen: Int)</code>
Solution section	Commented-out declarations for every task, in order
Incident reports	The broken code for Level 6, commented out

Everything else you write yourself. That includes every type.

Level 1 · The Deck Register (enums, raw values)

Task 1.1 — Deck (0.5 points)

```
enum Deck: String, CaseIterable
```

Five cases: `bridge`, `lab`, `cargo`, `medbay`, `engine`. Do **not** write raw values by hand — Swift assigns them.

Add a **read-only computed property**:

```
var evacuationPriority: Int // bridge 1, medbay 2, lab 3, engine 4, cargo 5
```

Implement it with a `switch` over `self`. No `default` case — let the compiler check that you covered everything.

Print every deck and its priority using `Deck.allCases`.

Task 1.2 — AlarmLevel (0.5 points)

```
enum AlarmLevel: Int // green = 0, then yellow, orange, red
```

Write `green = 0` only. The other three must get their raw values implicitly.

Add a type method (`static`):

```
static func level(forTotalMass mass: Int) -> AlarmLevel
```

The rule: every full 500 kg raises the alarm one step, and anything at or above 1500 kg is `red`. Build the result from `AlarmLevel(rawValue:)` — do not write a chain of `if` s comparing masses.

```
AlarmLevel.level(forTotalMass: 0) // green
AlarmLevel.level(forTotalMass: 940) // yellow
AlarmLevel.level(forTotalMass: 4000) // red
```

Level 2 · The Manifest (associated values)

Task 2.1 — ManifestEntry (0.5 points)

An enum with four cases, each carrying different associated values:

```
case crate(id: Int, massKg: Int)
case container(code: String, massKg: Int)
case livestock(species: String, count: Int, massPerUnitKg: Int)
case unknown(raw: String)
```

Task 2.2 — parseEntry(._) (1 point)

```
func parseEntry(_ line: String) -> ManifestEntry
```

The format is colon-separated:

Line	Result
"crate:101:120"	<code>.crate(id: 101, massKg: 120)</code>
"container:KZ-ALM-7:340"	<code>.container(code: "KZ-ALM-7", massKg: 340)</code>
"livestock:lab mice:12:2"	<code>.livestock(species: "lab mice", count: 12, massPerUnitKg: 2)</code>
anything else	<code>.unknown(raw: line)</code>

A line is corrupted if the tag is unknown, the field count is wrong, or a number doesn't parse. Never crash, never return an optional — `.unknown` **is** your error case. This is the enum equivalent of what `nil` did for you in Module 3.

Task 2.3 — mass(of:) (0.5 points)

```
func mass(of entry: ManifestEntry) -> Int
```

Switch over the entry and bind its associated values with `let`. Livestock mass is `count * massPerUnitKg`. Unknown entries weigh 0.

Parse all of `rawManifest`, sum the masses, and print how many lines were `.unknown`.

Integrity fragment A = the total mass of the manifest.

Level 3 · Crew Snapshots (structs and value semantics)

Task 3.1 — CrewSnapshot (0.5 points)

```
struct CrewSnapshot {  
    let name: String  
    var deck: Deck  
    var oxygen: Int  
}
```

Do **not** write an initializer — explain in the defense questions why you don't need one here but will need one in Level 4.

Add:

```
mutating func breathe(_ amount: Int) // lowers oxygen, never below 0  
mutating func move(to deck: Deck) // changes deck  
mutating func reviveInMedbay() // assigns a brand-new value to self  
static func rookie(named name: String) -> CrewSnapshot // deck .bridge, oxygen 100
```

`reviveInMedbay()` must assign a whole new `CrewSnapshot` to `self`, not set properties one by one.

Task 3.2 — Build the roster (0.5 points)

Turn `crewData` into `[CrewSnapshot]`. The deck comes from the string via `Deck(rawValue:)`; if a record names a deck that doesn't exist, skip that crew member and print a warning.

Task 3.3 — Prove it's a value type (0.5 points)

Write a demonstration that prints, with labels:

1. A copy of a snapshot modified — and the original unchanged.
2. A snapshot passed into a plain (non-`inout`) function that changes it — and the original unchanged.
3. The same call using `inout` — original changed.

Three lines of `print` are not enough. Each claim needs a before/after pair.

Level 4 · The Teleport Pod (classes and reference semantics)

Task 4.1 — TeleportPod (0.5 points)

```
final class TeleportPod {  
    let id: String  
    var chargeLevel: Int  
    var occupant: CrewSnapshot?  
}
```

Write the initializer yourself — `init(id: String, chargeLevel: Int)` — and set `occupant` to `nil`. Explain in a comment why the struct in Level 3 got its initializer for free and this class does not.

Methods:

```
func load(_ crew: CrewSnapshot) -> Bool // fails if the pod is occupied or charge < 20  
func fire() -> CrewSnapshot? // no occupant -> nil, and no charge is spent
```

A successful `fire()` costs **20** charge, empties the pod, and returns the occupant.

Task 4.2 — The charge ledger (0.5 points)

Create `TeleportPod(id: "P-1", chargeLevel: 100)` and run exactly this sequence:

1. `load Timur, fire()`
2. `load Dana, fire()`
3. `load Nurlan, fire()`
4. `fire()` again with an empty pod

Print the charge after every step.

Integrity fragment C = the pod's charge level after the whole sequence.

Task 4.3 — Prove it's a reference type (0.5 points)

Assign the pod to a second variable, change the charge through one of them, and print both. Then do the same experiment

with a `CrewSnapshot`. Write one comment sentence stating the rule you just demonstrated.

Level 5 · Station Systems (properties)

Task 5.1 — Station (1 point)

A class holding the station's live state. It must contain **all five** kinds of property from the lecture:

Kind	What to write
Stored (<code>let</code>)	<code>let callSign: String</code>
Stored (<code>var</code>) with observers	<code>var hullIntegrity: Int</code>
Lazy stored	<code>lazy var fullDiagnostics: String</code>
Computed, read-only	<code>var totalOxygen: Int</code>
Computed, get + set	<code>var averageOxygen: Int</code>

Details:

- `oxygenByDeck: [Deck: Int]` is a stored property, built from `deckReadings`. Readings for decks that don't exist are skipped.
- `totalOxygen` sums the dictionary's values. Read-only, and written **without** the `get` keyword.
- `averageOxygen` returns `totalOxygen / oxygenByDeck.count` (integer division). Its **setter sets every deck** to the new value. Use the default `newValue` name — don't name the setter parameter.
- `fullDiagnostics` prints `"Running full scan..."` while it builds its string. Prove in your output that this happens **on the first access only**, and never if the property is not touched.
- `hullIntegrity` has both observers: `willSet` prints the transition, `didSet` clamps the stored value into `0...100`.

Integrity fragment B = `averageOxygen` at start-up, before you change anything.

Task 5.2 — The clamp trap (0.5 points)

Set `hullIntegrity` to `130`, then to `-40`, then to `55`. Print it after each assignment and explain in a comment why the `didSet` clamp does not loop forever when it assigns to its own property.

Level 6 · Incident Reports (bug hunt, 1 point)

Four excerpts from the day-ten logs are in your starter file, commented out. **Three compile and are wrong. One doesn't compile at all.**

For each one: say what the author expected, what actually happens (or why it won't build), name the language rule behind it, and write the fixed version.

```
// Report 1
var roster = crewRoster
for var member in roster {
    member.oxygen -= 10
}
print(roster[0].oxygen) // author expected the crew to have lost oxygen

// Report 2
let podA = TeleportPod(id: "A", chargeLevel: 100)
let podB = podA
podB.chargeLevel = 0
print(podA.chargeLevel) // author expected 100

// Report 3
struct Logbook {
    var entries: [String] = []
    func add(_ entry: String) {
        entries.append(entry)
    }
}

// Report 4
let snapshot = CrewSnapshot.rookie(named: "Dana")
snapshot.oxygen = 40

let pod = TeleportPod(id: "B", chargeLevel: 50)
pod.chargeLevel = 10
```

Report 4 is the important one: both are declared with `let`, and only one of the two assignments is an error. Your explanation must say exactly what `let` protects in each case.

Level 7 · Sealing the Black Box (access control, 1 point)

The flight recorder currently lets anyone rewrite history:

```
class FlightRecorder {
    var entries: [String] = []
    var isSealed = false
}
```

Rewrite it so that, from outside the type, it is **impossible** to:

- replace or clear the entry list,
- append an entry after the recorder is sealed,
- change `isSealed` back to `false`.

It must still be possible, from outside, to read how many entries exist, read a formatted transcript, add an entry, and seal the recorder. Use explicit access-level keywords, a `private` stored property, and one `fileprivate` helper used by a free function elsewhere in the file. Write a one-line comment above each access keyword saying what it blocks.

Then try to break your own recorder from outside the type. Leave the failed attempt in a comment with the compiler error you got.

Finale: The Integrity Code

```
let integrityCode = "(A)-(B)-(C)-(D)"
print("INTEGRITY CODE: \(integrityCode)")
```

- **A** — total manifest mass (Task 2.3)
- **B** — starting `averageOxygen` (Task 5.1)
- **C** — pod charge after the sequence (Task 4.2)
- **D** — the `rawValue` of `AlarmLevel.level(forTotalMass: A)`

Every fragment must be computed by your code. Typed-in numbers are not accepted.

Defense Questions (0.5 points)

Answer as comments at the end of your file. Short, in your own words, with code where it helps.

1. Why did `CrewSnapshot` get an initializer for free while `TeleportPod` did not?
2. What does `mutating` actually do to `self`, and why do classes never need it?
3. In Report 4, both values are declared with `let`. Explain precisely what `let` freezes for a struct and for a class.
4. Why must a `lazy` property be `var`? Give a situation where making a property `lazy` changes program behaviour, not just performance.
5. `private` vs `fileprivate`: describe a concrete case in your `FlightRecorder` where `private` would be too strict and `fileprivate` is the right level.

Bonus (+1 point, explore on your own)

The lecture mentions two class-only features we didn't use: **deinitializers** and **reference counting**.

1. Add `deinit` to `TeleportPod` that prints the pod's id.
 2. Create a pod inside a `do { }` block, keep a second reference to it, and print messages before and after leaving the block. Show which line the `deinit` fires on, and explain why.
 3. The station now has two records for one crew member. Using the identity operator `===`, write a function that decides whether two references point to the **same** pod or to two pods with equal contents. Then explain why `===` cannot be used on `CrewSnapshot` at all.
-

Grading

Part	Points
Level 1 — Deck Register	1
Level 2 — Manifest	2
Level 3 — Crew Snapshots	1.5
Level 4 — Teleport Pod	1.5
Level 5 — Station Systems	1.5
Level 6 — Incident Reports	1
Level 7 — Black Box	1
Defense questions	0.5
Total	10
Bonus	+1
Each ! outside Level 6	−0.5
Wrong integrity code	−1
A <code>class</code> where a <code>struct</code> would do, with no justification	−0.5

The station is stable. Keep it that way.